

How Narbis Captures and Calculates HRV on Apple Watch

This document traces a single heartbeat from the Apple Watch sensor through to the coherence percentage displayed on screen. Every function referenced here is production code in the Narbis codebase.

1. Capturing Raw Heartbeats from HealthKit

The Apple Watch uses its optical heart rate sensor to detect individual heartbeats. HealthKit exposes these through `HKHeartbeatSeriesSample` — a time-series of beat timestamps at sub-millisecond resolution.

`HKHealthStoreAdapter.processHeartbeatSeries()` (`BioFeedbackKit-HealthKit`)

When a workout session is active, the adapter runs an anchored object query on `HKSeriesType.heartbeat()`. As each series sample arrives, it enumerates the individual beat timestamps and computes **RR intervals** — the time between successive heartbeats:

```
RR interval (ms) = (beatTimestamp[i+1] - beatTimestamp[i]) × 1000
```

For example, if two beats occur at t=1.234s and t=2.056s, the RR interval is 822 ms, corresponding to a heart rate of 60000/822 = 73 BPM.

A physiologic range gate rejects intervals outside 300–1500 ms (corresponding to 40–200 BPM). This eliminates sensor artifacts like motion noise or optical crosstalk.

If heartbeat series data isn't available (older watchOS, certain conditions), the adapter falls back to `HKQuantityType.heartRate()` samples and derives RR as 60000 / BPM. This is lower resolution (1 Hz vs. beat-by-beat) but sufficient for basic HRV.

Each valid RR interval becomes a `BioSample` — the universal sample type across all Narbis device adapters:

```
BioSample{rrInterval: 822.0, // milliseconds
  timestamp: /* ContinuousClock.Instant */}
```

2. Filtering Ectopic Beats

Not every RR interval represents a real heartbeat. Premature ventricular contractions (PVCs), sensor dropouts, and motion artifacts produce outlier intervals that would corrupt HRV calculations. The `RRBuffer` applies two filters before any sample reaches the algorithm.

Stage 1: Range Gate

Rejects any RR interval outside 300–2000 ms. This catches gross artifacts (e.g., a sensor dropout producing a 5000 ms gap).

Stage 2: Median Malik Filter (`EctopicFilter`)

The default filter maintains a sliding window of the 5 most recently accepted beats. A new sample is rejected if it differs from the median by more than 20%:

```
|newRR - median(last5)| / median(last5) > 0.20 → reject
```

This is more robust than comparing to just the previous beat, because a burst of 2-3 corrupt beats won't shift the median. The filter is based on the Malik correction method from the European Heart Journal guidelines.

3. Time-Domain HRV: RMSSD and SDNN

Once filtered, samples feed into `HRVMetrics`, which computes the standard time-domain HRV statistics from any window of RR intervals.

`HRVMetrics.init(window:)` (`BioFeedbackKit/Signal`)

RMSSD (Root Mean Square of Successive Differences):

```
successive_diffs = [RR[1]-RR[0], RR[2]-RR[1], ..., RR[n-1]-RR[n-2]]
RMSSD = √(mean(diff² for each diff))
```

RMSSD reflects beat-to-beat variability and is dominated by parasympathetic (vagal) activity. Higher RMSSD = more vagal tone = more relaxed. This is the most widely used short-term HRV metric and the one that responds fastest to breathing patterns.

SDNN (Standard Deviation of NN intervals):

```
SDNN = sample_std_dev(all RR intervals in window)
```

SDNN reflects total variability — both sympathetic and parasympathetic. It requires longer windows (ideally 5 minutes) to be clinically meaningful. We compute it for telemetry but don't use it for coherence scoring.

pNN50 (percentage of successive differences > 50 ms):

```
pNN50 = count(|diff| > 50ms) / count(diffs)
```

The 50 ms threshold is physiological, not mathematical — it represents the approximate boundary between normal sinus variation and clinically significant variability. We include it in `HRVReport` telemetry.

4. Frequency-Domain Analysis: Finding the Breathing Peak

The coherence algorithm needs to know where in the frequency spectrum the heart rate variability is concentrated. This requires transforming the irregular RR time series into the frequency domain.

`FrequencyDomainMetrics.init(window:)` (`BioFeedbackKit/Signal`)

Step 1: Resampling

RR intervals arrive at irregular times (because heartbeats aren't evenly spaced). FFT requires evenly-spaced samples. The pipeline resamples the RR series onto a uniform 4 Hz grid using **natural cubic spline interpolation** (the Kubios standard). This preserves the shape of the RR curve far better than linear interpolation — amplitude error drops from ~33% to ~3% in the HF band.

Step 2: Preprocessing

Mean removal subtracts the average RR from every sample, eliminating the DC component (which would dominate the spectrum but carries no variability information).

Step 3: Windowing

A Hann window tapers the edges of the signal to zero, reducing spectral leakage from the FFT:

```
w[n] = 0.5 × (1 - cos(2π × n / (N-1)))
```

Step 4: FFT and Power Spectral Density

The windowed signal passes through an FFT (vDSP/Accelerate on Apple platforms). The result is converted to a one-sided power spectral density (PSD) in units of ms²/Hz. The Hann window's noise-equivalent bandwidth is compensated so that integrated band power matches the true signal power.

Step 5: Band Integration

The PSD is integrated across standard physiological bands:

Band	Range	Physiological meaning
VLF	0.003–0.04 Hz	Thermoregulation, hormonal (requires >5 min window)
LF	0.04–0.15 Hz	Mix of sympathetic + parasympathetic
HF	0.15–0.40 Hz	Parasympathetic / respiratory sinus arrhythmia

`band_power = Σ(PSD[k] × Δf)` for all k where `f[k]` is in [low, high)

The **LF/HF ratio** is also computed, though the coherence algorithm uses a different approach.

5. Coherence Scoring: How Concentrated Is Your Breathing Peak?

Coherence measures how much of the heart rate variability is organized around a single frequency — your breathing rate. When you breathe rhythmically at ~6 breaths per minute (0.1 Hz), the PSD develops a sharp, dominant peak. When your breathing is irregular or your autonomic nervous system is fragmented, the power spreads across many frequencies.

`CoherenceAlgorithm.score()` (`BioFeedbackKit/Algorithm`)

Step 1: Find the dominant peak in the breathing band [0.04, 0.26] Hz:

```
peak = argmax(PSD[k]) for k where 0.04 ≤ f[k] < 0.26
```

The upper boundary of 0.26 Hz (15.6 breaths/min) covers the full range of normal breathing rates for coherence training. The lower boundary of 0.04 Hz matches the LF band floor.

Step 2: Measure peak concentration. Integrate PSD power in a narrow window around the peak (±0.015 Hz) and compare to total power in the breathing band:

```
numerator = ∫ PSD(f) df for f in [peak - 0.015, peak + 0.015]
denominator = ∫ PSD(f) df for f in [0.04, 0.26]
coherence = clamp(100 × numerator / denominator, 0, 100)
```

If 80% of the breathing-band power falls within the ±15 mHz peak window, coherence = 80%. This means the heart's rhythm is highly synchronized with the breathing pattern — the hallmark of physiological coherence.

Why ±0.015 Hz? This window width (30 mHz total) is wide enough to capture the main lobe of a breathing-rate peak (which has finite width from the 60-second analysis window) but narrow enough to reject energy from adjacent frequencies. It's derived from the v5 algorithm specification.

6. Streaming Engine: Real-Time Smoothing

The `StreamingCoherenceEngine` actor manages the rolling buffer and produces one coherence score per incoming RR sample, after a warmup period.

`StreamingCoherenceEngine.addSample()` (`BioFeedbackKit/Algorithm`)

60-second rolling buffer: The engine maintains the most recent 60 seconds of accepted `BioSamples`. Older samples are trimmed automatically on each arrival. This window is long enough for meaningful frequency resolution (minimum ~25 seconds for LF-band analysis) but short enough to respond to changes in breathing pattern within a minute.

30-second warmup gate: No coherence score is emitted until the buffer spans at least 30 seconds. This prevents the algorithm from producing meaningless scores from insufficient data during the first half-minute of a session.

EMA smoothing (α = 0.3): Raw coherence scores jump between samples because each new RR interval shifts the FFT window. An exponential moving average smooths the displayed value:

```
smoothed_t = 0.3 × raw_t + 0.7 × smoothed_{t-1}
```

With α = 0.3, the effective time constant is about 3 samples (roughly 3 seconds at resting heart rate). This is fast enough to feel responsive but slow enough to avoid distracting jitter. The smoothed value is what drives the coherence ring, the percentage display, and the glasses opacity.

7. Display and Feedback

The final `CoherenceResult` carries:

Field	Type	What it means
coherence	0–100%	EMA-smoothed coherence score
peakFrequency	Hz	Dominant peak in breathing band
breathingRate	BPM	peakFrequency × 60
rmssd	ms	Beat-to-beat variability
lfPower	ms²/Hz	Low-frequency power
hfPower	ms²/Hz	High-frequency power
lfHfRatio	ratio	Sympathovagal balance indicator

In `TrainingViewModel`, the smoothed coherence drives: - **Coherence ring** — fills and changes color (red < 30%, yellow 30–60%, green > 60%) - **Percentage text** — large centered display - **Edge glasses brightness** — via `TintMapper` (higher coherence = clearer lenses) - **Audio warmth** — harmonic richness modulates from coherence when adaptive harmonics are enabled - **Session summary** — coherence time series recorded for post-session review

The breathing circle animation runs independently of the coherence calculation, driven by `BreathingPacer` at the configured breathing rate (typically 6 BPM). The circle provides the visual cue for the user to match their breathing to; the coherence score tells them how well they're doing.